

stats (version 3.6.2)

smooth.spline: Fit a Smoothing Spline

Description

Fits a cubic smoothing spline to the supplied data.

Usage

```
smooth.spline(x, y = NULL, w = NULL, df, spar = NULL, lambda = NULL, cv = FALSE,  
              all.knots = FALSE, nknots = .nknots.smspl,  
              keep.data = TRUE, df.offset = 0, penalty = 1,  
              control.spar = list(), tol = 1e-6 * IQR(x), keep.stuff = FALSE)
```

Arguments

x

The State of Data & AI Literacy 2024

DOWNLOAD NOW

responses. If `y` is missing or `NULL`, the responses are assumed to be specified by `x`, with `x` the index vector.

w

optional vector of weights of the same length as `x`; defaults to all 1.

df

the desired equivalent number of degrees of freedom (trace of the smoother matrix). Must be in $(1, n_x]$, n_x the number of unique x values, see below.

spar

smoothing parameter, typically (but not necessarily) in $(0, 1]$. When `spar` is specified, the coefficient λ of the integral of the squared second derivative in the fit (penalized log likelihood) criterion is a monotone function of `spar`, see the

details below. Alternatively ``lambda`` may be specified instead of the *scale free* ``spar` = s`.

`lambda`

if desired, the internal (design-dependent) smoothing parameter λ can be specified instead of ``spar``. This may be desirable for resampling algorithms such as cross validation or the bootstrap.

`cv`

ordinary leave-one-out (``TRUE``) or ‘generalized’ cross-validation (GCV) when ``FALSE``; is used for smoothing parameter computation only when both ``spar`` and ``df`` are not specified; it is used however to determine ``cv.crit`` in the result. Setting it to ``NA`` for speedup skips the evaluation of leverages and any score.

`all.knots`

if ``TRUE``, all distinct points in ``x`` are used as knots. If ``FALSE`` (default), a subset of ``x[]`` is used, specifically ``x[j]`` where the ``nknots`` indices are evenly spaced in ``1:n``, see also the next argument ``nknots``.

Alternatively, a strictly increasing ``numeric`` vector specifying “all the knots” to be used; must be rescaled to $[0, 1]$ already such that it corresponds to the ``ans $ fit$nknots`` sequence returned, not repeating the boundary knots.

`nknots`

integer or ``function`` giving the number of knots to use when ``all.knots = FALSE``. If a function (as by default), the number of knots is ``nknots(nx)``. By default for $n_x > 49$ this is less than n_x , the number of unique ``x`` values, see the Note.

`keep.data`

logical specifying if the input data should be kept in the result. If ``TRUE`` (as per default), fitted values and residuals are available from the result.

`df.offset`

allows the degrees of freedom to be increased by ``df.offset`` in the GCV criterion.

`penalty`

the coefficient of the penalty for degrees of freedom in the GCV criterion.

`control.spar`

optional list with named components controlling the root finding when the smoothing parameter ``spar`` is computed, i.e., missing or ``NULL``, see below.

Note that this is partly *experimental* and may change with general `spar` computation improvements!

low:

lower bound for ``spar``; defaults to -1.5 (used to implicitly default to 0 in R versions earlier than 1.4).

high:

upper bound for ``spar``; defaults to +1.5.

`tol`:

the absolute precision (**tolerance**) used; defaults to $1e-4$ (formerly $1e-3$).

`eps`:

the relative precision used; defaults to $2e-8$ (formerly 0.00244).

`trace`:

logical indicating if iterations should be traced.

`maxit`:

integer giving the maximal number of iterations; defaults to 500.

Note that ``spar`` is only searched for in the interval $[low, high]$.

tol

a tolerance for same-ness or uniqueness of the ``x`` values. The values are binned into bins of size ``tol`` and values which fall into the same bin are regarded as the same. Must be strictly positive (and finite).

keep.stuff

an experimental ``logical`` indicating if the result should keep extras from the internal computations. Should allow to reconstruct the X matrix and more.

Value

An object of class ``"smooth.spline"`` with components

x	the <i>distinct</i> <code>`x`</code> values in increasing order, see the ‘Details’ above.
y	the fitted values corresponding to <code>`x`</code> .
w	the weights used at the unique values of <code>`x`</code> .
yin	the y values used at the unique <code>`y`</code> values.
tol	the <code>`tol`</code> argument (whose default depends on <code>`x`</code>).
data	only if <code>`keep.data = TRUE`</code> : itself a <code>`list`</code> with components <code>`x`</code> , <code>`y`</code> and <code>`w`</code> of the same length. These are the original $(x_i, y_i, w_i), i = 1, \dots, n$, values where <code>`data\$x`</code> may have repeated values and hence be longer than the above <code>`x`</code> component; see details.
lev	(when <code>`cv`</code> was not <code>`NA`</code>) leverages, the diagonal values of the smoother matrix.

cv.crit	cross-validation score, ‘generalized’ or true, depending on <code>cv</code> . The CV score is often called “PRESS” (and labeled on <code>print()</code>), for ‘ P REdiction S um of S quares’.								
pen.crit	the penalized criterion, a non-negative number; simply the (weighted) residual sum of squares (RSS), <code>sum(.w * residuals())^2</code> .								
crit	the criterion value minimized in the underlying <code>.Fortran</code> routine <code>sslvrg</code> . When <code>df</code> has been specified, the criterion is $3 + (tr(S_\lambda) - df)^2$, where the 3+ is there for numerical (and historical) reasons.								
df	equivalent degrees of freedom used. Note that (currently) this value may become quite imprecise when the true <code>df</code> is between and 1 and 2.								
spar	the value of <code>spar</code> computed or given, unless it has been given as <code>c(lambda = *)</code> , when it set to <code>NA</code> here.								
ratio	(when <code>spar</code> above is not <code>NA</code>), the value r , the ratio of two matrix traces.								
lambda	the value of λ corresponding to <code>spar</code> , see the details above.								
iparms	named integer(3) vector where <code>..\$ipars["iter"]</code> gives number of spar computing iterations used.								
auxMat	experimental; when <code>keep.stuff</code> was true, a “flat” numeric vector containing parts of the internal computations.								
fit	list for use by <code>predict.smooth.spline</code> , with components <table> <tr> <td>knot:</td><td>the knot sequence (including the repeated boundary knots), scaled into $[0, 1]$ (via <code>min</code> and <code>range</code>).</td></tr> <tr> <td>nk:</td><td>number of coefficients or number of ‘proper’ knots plus 2.</td></tr> <tr> <td>coef:</td><td>coefficients for the spline basis used.</td></tr> <tr> <td>min, range:</td><td>numbers giving the corresponding quantities of <code>x</code>.</td></tr> </table>	knot:	the knot sequence (including the repeated boundary knots), scaled into $[0, 1]$ (via <code>min</code> and <code>range</code>).	nk:	number of coefficients or number of ‘proper’ knots plus 2.	coef:	coefficients for the spline basis used.	min, range:	numbers giving the corresponding quantities of <code>x</code> .
knot:	the knot sequence (including the repeated boundary knots), scaled into $[0, 1]$ (via <code>min</code> and <code>range</code>).								
nk:	number of coefficients or number of ‘proper’ knots plus 2.								
coef:	coefficients for the spline basis used.								
min, range:	numbers giving the corresponding quantities of <code>x</code> .								
call	the matched call.								

`method(class = "smooth.spline")` shows a [hatvalues\(\)](#) method based on the lev vector above.

Details

Neither `x` nor `y` are allowed to containing missing or infinite values.

The `x` vector should contain at least four distinct values. ‘Distinct’ here is controlled by `tol`: values which are regarded as the same are replaced by the first of their values and the corresponding `y` and `w` are pooled accordingly.

Unless `lambda` has been specified instead of `spar`, the computational λ used (as a function of $s = spar$) is $\lambda = r * 256^{3s-1}$ where $r = tr(X'WX)/tr(\Sigma)$, Σ is the matrix given by $\Sigma_{ij} = \int B_i''(t)B_j''(t)dt$, X is given by $X_{ij} = B_j(x_i)$, W is the diagonal matrix of weights (scaled such that its trace is n , the original number of observations) and $B_k(\cdot)$ is the k -th B-spline.

Note that with these definitions, $f_i = f(x_i)$, and the B-spline basis representation $f = Xc$ (i.e., c is the vector of spline coefficients), the penalized log likelihood is $L = (y - f)'W(y - f) + \lambda c'\Sigma c$, and hence c is the solution of the (ridge regression) $(X'WX + \lambda\Sigma)c = X'Wy$.

If `spar` and `lambda` are missing or `NULL`, the value of `df` is used to determine the degree of smoothing. If `df` is missing as well, leave-one-out cross-validation (ordinary or ‘generalized’ as determined by `cv`) is used to determine λ .

Note that from the above relation, `spar` is $s = s0 + 0.0601 * \log \lambda$, which is intentionally *different* from the S-PLUS implementation of `smooth.spline` (where `spar` is proportional to λ). In R's ($\log \lambda$) scale, it makes more sense to vary `spar` linearly.

Note however that currently the results may become very unreliable for `spar` values smaller than about -1 or -2. The same may happen for values larger than 2 or so. Don't think of setting `spar` or the controls `low` and `high` outside such a safe range, unless you know what you are doing! Similarly, specifying `lambda` instead of `spar` is delicate, notably as the range of “safe” values for `lambda` is not scale-invariant and hence entirely data dependent.

The ‘generalized’ cross-validation method GCV will work correctly when there are duplicated points in `x`. However, it is ambiguous what leave-one-out cross-validation means with duplicated points, and the internal code uses an approximation that involves leaving out groups of duplicated points. `cv = TRUE` is best avoided in that case.

References

Chambers, J. M. and Hastie, T. J. (1992) *Statistical Models in S*, Wadsworth & Brooks/Cole.

Green, P. J. and Silverman, B. W. (1994) *Nonparametric Regression and Generalized Linear Models: A Roughness Penalty Approach*. Chapman and Hall.

Hastie, T. J. and Tibshirani, R. J. (1990) *Generalized Additive Models*. Chapman and Hall.

See Also

`predict.smooth.spline` for evaluating the spline and its derivatives.

Examples

[Run this code](#)

```
# NOT RUN {
require(graphics)
plot(dist ~ speed, data = cars, main = "data(cars) & smoothing splines")
cars.spl <- with(cars, smooth.spline(speed, dist))
cars.spl
## This example has duplicate points, so avoid cv = TRUE
# }
# NOT RUN {
lines(cars.spl, col = "blue")
ss10 <- smooth.spline(cars[, "speed"], cars[, "dist"], df = 10)
lines(ss10, lty = 2, col = "red")
legend(5, 120, c(paste("default [C.V.] => df =", round(cars.spl$df, 1)),
  "s( * , df = 10)"), col = c("blue", "red"), lty = 1:2,
  bg = 'bisque')

## Residual (Tukey Anscombe) plot:
plot(residuals(cars.spl) ~ fitted(cars.spl))
abline(h = 0, col = "gray")

## consistency check:
stopifnot(all.equal(cars$dist,
  fitted(cars.spl) + residuals(cars.spl)))

## The chosen inner knots in original x-scale :
with(cars.spl$fit, min + range * knot[-c(1:3, nk+1 +1:3)]) # == unique(cars$speed)

## Visualize the behavior of .nknots.smspl()
nKnots <- Vectorize(.nknots.smspl) ; c.. <- adjustcolor("gray20", .5)
curve(nKnots, 1, 250, n=250)
abline(0, 1, lty=2, col=c..); text(90, 90, "y = x", col=c.., adj=-.25)
abline(h=100, lty=2); abline(v=200, lty=2)

n <- c(1:799, seq(800, 3490, by=10), seq(3500, 10000, by = 50))
plot(n, nKnots(n), type="l", main = "Vectorize(.nknots.smspl) (n)")
abline(0, 1, lty=2, col=c..); text(180, 180, "y = x", col=c..)
n0 <- c(50, 200, 800, 3200); c0 <- adjustcolor("blue3", .5)
lines(n0, nKnots(n0), type="h", col=c0)
axis(1, at=n0, line=-2, col.ticks=c0, col=NA, col.axis=c0)
axis(4, at=.nknots.smspl(10000), line=-.5, col=c.., col.axis=c.., las=1)

##-- artificial example
y18 <- c(1:3, 5, 4, 7:3, 2*(2:5), rep(10, 4))
xx <- seq(1, length(y18), len = 201)
(s2 <- smooth.spline(y18)) # GCV
(s02 <- smooth.spline(y18, spar = 0.2))
(s02. <- smooth.spline(y18, spar = 0.2, cv = NA))
plot(y18, main = deparse(s2$call), col.main = 2)
lines(s2, col = "gray"); lines(predict(s2, xx), col = 2)
```

```
lines(predict(s02, xx), col = 3); mtext(deparse(s02$call), col = 3)

## Specifying 'lambda' instead of usual spar :
(s2. <- smooth.spline(y18, lambda = s2$lambda, tol = s2$tol))

# }
# NOT RUN {
## The following shows the problematic behavior of 'spar' searching:
(s2 <- smooth.spline(y18, control =
  list(trace = TRUE, tol = 1e-6, low = -1.5)))
(s2m <- smooth.spline(y18, cv = TRUE, control =
  list(trace = TRUE, tol = 1e-6, low = -1.5)))
## both above do quite similarly (Df = 8.5 +- 0.2)
# }
```

Run the code above in your browser using [DataLab](#)

Powered by [DataCamp](#)